Introduction to Computer Architecture

Sheet #6

Exercises:

1. Write the following code segment in MARIE's assembly language:

```
if X > 1 then
   Y := X + X;
   X := 0;
endif;
Y := Y + 1;
```

ANSWER:

```
If,     100   Load      X       /Load X
        101   Subt      One     /Subtract 1, store result in AC
        102   Skipcond  800     /If AC>0 (X>1), skip the next instruction
        103   Jump      Endif   /Jump to Endif if X is not greater than 1
Then,   104   Load      X       /Reload X so it can be doubled
        105   Add       X       /Double X
        106   Store     Y       /Y:= X + X
        107   Clear             /Move 0 into AC
        108   Store     X       /Set X to 0
Endif,  109   Load      Y       /Load Y into AC
        10A   Add       One     /Add 1 to Y
        10B   Store     Y       /Y := Y + 1
        10C   Halt              /Terminate program
X,      10D   Dec       ?       /X has starting value, not given in problem
Y,      10E   Dec       ?       /Y has starting value, not given in problem
One,    110   Dec       1       /Use as a constant
```

2. What are the potential problems (perhaps more than one) with the following assembly language code fragment (implementing a subroutine) written to run on MARIE? The subroutine assumes the parameter to be passed is in the AC and should double this value. The Main part of the program includes a sample call to the subroutine. You can assume this fragment is part of a larger program.

```
Main,   Load   X
        Jump   Sub1
Sret,   Store  X
    …
Sub1,   Add    X
        Jump   Sret
```

ANSWER:

First, this subroutine works only for the parameter X (no other variable could be used as X is explicitly added in the subroutine). Second, this subroutine cannot be called from anywhere, as it always returns to Sret.

3. Write a MARIE program to evaluate the expression A x B + C x D.

ANSWER:

```
        ORG     100
        Load    A
        Store   X       /Store A in first parameter
        Load    B
        Store   Y       /Store B in second parameter
        JnS     Mul     /Jump to multiplication subroutine
        Load    Sum     /Get result
        Store   E       /E:= A x B
        Load    C
        Store   X       /Store C in first parameter
        Load    D
        Store   Y       /Store D in second parameter
        JnS     Mul     /Jump to multiplication subroutine
        Load    Sum     /Get result
        Store   F       /F := C x D
        Load    E       /Get first result
        Add     F       /AC now contains sum of A X B + C X D
        Halt            /Terminate program
A,      Dec     ?       /Initial values of A,B,C,D not given in problem
B,      Dec     ?       / (give values before assembling and running)
C,      Dec     ?       /
D,      Dec     ?       /
X,      Dec     0       /First parameter
Y,      Dec     0       /Second parameter
Ctr,    Dec     0       /Counter for looping
One,    Dec     1       /Constant with value 1
E,      Dec     0       /Temp storage
F,      Dec     0       /Temp storage
Sum,    Dec     0
Mul,    Hex     0       /Store return address here
        Load    Y       /Load second parameter to be used as counter
        Store   Ctr     /Store as counter
        Clear           /Clear sum
        Store   Sum     /Zero out the sum to begin
Loop,   Load    Sum     /Load the sum
        Add     X       /Add first parameter
        Store   Sum     /Store result in Sum
        Load    Ctr
        Subt    One     /Decrement counter
        Store   Ctr     /Store counter
        SkipCond 400    /If counter = 0 finish subroutine
        Jump    Loop    /Continue subroutine loop
        JumpI   Mul     /Done with subroutine, return to main
        END
```

4. Write a MARIE subroutine to subtract two numbers.
   ANSWER:
   Assume the formal parameters are X and Y, and we are subtracting Y from X. Assume also that the actual parameters are A and B. These values could be input or declared in the program. This program tests the subroutine with two sets of values.

```
         ORG      100
         Load     A        /Load the first number
         Store    X        /Let X be the first parameter
         Load     B        /Load the second number
         Store    Y        /Let Y be the second parameter
         JnS      Subr     /Store return address, jump to procedure
         Load     X        /Load the result
         Output            /Output the first difference
         Load     C
         Store    X
         Load     D
         Store    Y
         JnS      Subr
         Load     X
         Output            /Output the second difference
         Halt              /Terminate program
X,       Dec      0        /These could also be input
Y,       Dec      0
A,       Dec      8        /A and B could be input or declared
B,       Dec      4
C,       Dec      10
D,       Dec      2
Subr,    Hex      0        /Store return address here
         Load     X        /Load the first number
         Subt     Y        /Subtract the second number
         Store    X        /Store result in first parameter
         JumpI    Subr
         END
```

5. Given a pointer to a number, double that number and store the result back using only indirect addressing.
   ANSWER:

```
ORG 100
Clear
LoadI AI
AddI AI
StoreI AI
Output
Halt

A, Dec 3
AI, Hex 106
```

6. You're given a number to store in memory, write a program that:
   a. Loads the number into the accumulator.
   b. Calls a subroutine to double the value.
   c. The subroutine multiplies the value by 2 using basic arithmetic.
   d. The result is stored back in the original memory location.

e. Control returns to the main program after the subroutine finishes.
ANSWER:

```
ORG 100
Input
Store A
Store X
JnS multiplication
Load X
Output
Halt

multiplication, hex 0
Load X
Add X
Store X
JumpI multiplication

A, DEC 0
X, DEC 0
```